One way of doing this is to use the standard Resource Acquisition Is Initialization (RAII) idiom and provide a class that does the `join()` in its destructor, as in the following listing. See how it simplifies the function `f()`.

Listing 2.3. Using RAII to wait for a thread to complete

```
class thread_guard
{
    std::thread& t;
public:
    explicit thread_guard(std::thread& t_):
        t(t_)
    {}
    ~thread_guard()
    {
        if(t.joinable())          <-1
        {
            t.join();             <-2
        }
    }
    thread_guard(thread_guard const&)=delete;                    <-3
    thread_guard& operator=(thread_guard const&)=delete;
};

struct func;                                  See definition
                                              in listing 2.1
void f()
{
    int some_local_state=0;
    func my_func(some_local_state);
    std::thread t(my_func);
    thread_guard g(t);

    do_something_in_current_thread();
}                                             <-4
```

When the execution of the current thread reaches the end of `f` ❹, the local objects are destroyed in reverse order of construction. Consequently, the `thread_guard` object `g` is destroyed first, and the thread is joined with in the destructor ❷. This even happens if the function exits because `do_something_in_current_thread` throws an exception.